

Fișa de lucru

Funcții în Python

Definire | Parametri | Return | Variabile locale și globale

Competențe vizate

1. Definirea și apelul funcțiilor în Python.
2. Utilizarea parametrilor și a valorilor returnate.
3. Înțelegerea domeniului de vizibilitate al variabilelor (local vs. global).
4. Organizarea și modularizarea codului folosind funcții.

1. Noțiuni teoretice

1.1 Ce este o funcție și de ce o folosim

Funcția

O funcție este un bloc de instrucțiuni cu un nume, care poate fi apelat ori de câte ori este nevoie. Funcțiile permit reutilizarea codului, reduc redundanța și împart un program complex în subprobleme mai simple (programare structurată / modulară).

Motivația utilizării funcțiilor:

Fără funcții, același cod trebuie rescris de fiecare dată când este necesar. Orice modificare trebuie făcută în toate locurile unde apare codul. Cu funcții, scriem codul o singură dată, îl apelăm de câte ori avem nevoie și orice modificare se face într-un singur loc.

1.2 Definirea și apelul unei funcții

```
# Sintaxa generală:
def nume_funcție(parametri):    # antet: def + nume + (parametri) + :
    # instrucțiuni            # corp: indentat cu 4 spații
    return valoare              # opțional: valoarea returnată

# O funcție poate avea ORICÂȚI parametri: zero, unul sau mai mulți.
def saluta():                  # niciun parametru
    print('Salut!')

def patrat(x):                 # un singur parametru
    return x * x

def suma(a, b, c):             # trei parametri
    return a + b + c
```

Numărul de parametri nu este fix: o funcție poate avea zero, unul sau mai mulți parametri, separați prin virgulă.

Apelul funcției — nu doar prin atribuire:

```
rezultat = patrat(5)          # 1) atribuim valoarea returnată unei variabile
print(patrat(5))              # 2) folosim direct valoarea returnată (ex. în print)
saluta()                      # 3) apel simplu, ca instrucțiune (fără atribuire)
suma(2, 3, 4)                 # apelul are tot atâtea argumente câți parametri
```

Distincție importantă: parametrii sunt variabilele din definiția funcției, iar argumentele sunt valorile concrete transmise la apel. Exemplu: `def f(x, y)` — `x` și `y` sunt parametri; `f(3, 5)` — `3` și `5` sunt argumente.

1.3 Instrucțiunea return

Instrucțiunea `return` încheie execuția funcției și trimite o valoare înapoi către locul apelului. O funcție poate returna orice tip de valoare (număr, șir, listă, etc.) sau poate să nu returneze nimic (funcție de tip procedură — returnează implicit `None`).

```
# Funcție care returnează o valoare:
def patrat(n):
    return n * n

x = patrat(5)      # x = 25
print(patrat(3))   # afișează 9

# Funcție care nu returnează valoare (procedură):
def afiseaza_linie(n):
    print('-' * n)

afiseaza_linie(20) # afișează: -----
```

1.4 Variabile locale și variabile globale

Tip variabilă	Definiție	Vizibilitate	Durată de viață
Locală	Declarată în interiorul unei funcții	Doar în interiorul funcției respective	De la intrarea în funcție până la ieșire
Globală	Declarată în afara oricărei funcții	În tot programul	Pe toată durata execuției programului

```
x = 10 # variabilă GLOBALĂ

def demonstratie():
    y = 5 # variabilă LOCALĂ — nu există în afara funcției
    print(x) # poate citi variabila globală
    # x = 20 # EROARE fără 'global'!

def modifica_globala():
    global x # declarare explicită obligatorie
    x = 20 # acum poate modifica variabila globală
```

Bună practică: evitați utilizarea variabilelor globale în funcții. Transmiteți datele prin parametri și primiți rezultatele prin `return`. Codul devine mai clar, mai ușor de înțeles și de depanat.

2. Exemple rezolvate

Exemplul 1 — Funcții pentru calcule matematice simple

Se definesc funcții pentru calculul ariei diferitelor figuri geometrice și se apelează în funcție de alegerea utilizatorului.

```

import math

def aria_cerc(r):
    return math.pi * r * r

def aria_dreptunghi(L, l):
    return L * l

def aria_triunghi(b, h):
    return (b * h) / 2

# Apelul funcțiilor
r = float(input('Raza cercului: '))
print('Aria cercului:', round(aria_cerc(r), 4))
L = float(input('Lungimea dreptunghiului: '))
l = float(input('Lățimea dreptunghiului: '))
print('Aria dreptunghiului:', aria_dreptunghi(L, l))

```

Exemplu de execuție (r=5, L=6, l=4):

```

Aria cercului: 78.5398
Aria dreptunghiului: 24.0

```

Exemplul 2 — Funcție care prelucrează o listă

Se definesc funcții separate pentru citirea unei liste, calculul statisticilor și afișarea rezultatelor. Programul principal orchestrează apelurile.

```

def citeste_lista(n):
    v = []
    for i in range(n):
        x = int(input(' v[' + str(i) + '] = '))
        v.append(x)
    return v

def statistici(v):
    return sum(v), min(v), max(v), sum(v)/len(v)

def afiseaza_statistici(suma, minim, maxim, medie):
    print('Suma: ', suma)
    print('Minim: ', minim)
    print('Maxim: ', maxim)
    print('Medie: ', round(medie, 2))

# Programul principal
n = int(input('Numărul de elemente: '))
v = citeste_lista(n)
s, mn, mx, med = statistici(v)
afiseaza_statistici(s, mn, mx, med)

```

Observație: funcția `statistici()` returnează patru valori simultan, separate prin virgulă (un tuplu). La apel, le descompunem în variabile separate: `s`, `mn`, `mx`, `med`.

Exemplul 3 — Funcție recursivă — calculul factorialului

Se calculează factorialul unui număr `n` folosind două abordări: iterativă și recursivă.

Varianta iterativă:

```

def factorial_iterativ(n):
    rezultat = 1
    for i in range(2, n + 1):
        rezultat *= i
    return rezultat

```

Varianta recursivă:

```
def factorial_recursiv(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursiv(n - 1) # apel recursiv

# Testare
for i in range(6):
    print(i, '!' '=', factorial_iterativ(i))
```

Exemplu de execuție:

```
0 ! = 1
1 ! = 1
2 ! = 2
3 ! = 6
4 ! = 24
5 ! = 120
```

3. Exerciții

1 Ușor Definiți o funcție `maxim2(a, b)` care returnează maximul a două numere, fără a folosi funcția predefinită `max()`. Definiți o funcție `maxim3(a, b, c)` care apelează `maxim2` și returnează maximul a trei numere.

2 Ușor Definiți o funcție `este_par(n)` care returnează `True` dacă `n` este par și `False` altfel. Definiți o funcție `este_prim(n)` care verifică dacă `n` este un număr prim. Afișați toate numerele prime de la 1 la 100 folosind aceste funcții.

Indiciu pentru `este_prim`: un număr este prim dacă nu are niciun divizor de la 2 la `sqrt(n)`. Puteți folosi `n**0.5` sau `math.sqrt(n)`.

3 Mediu Definiți o funcție `cmmdc(a, b)` care calculează cel mai mare divizor comun al numerelor `a` și `b`, folosind algoritmul lui Euclid: dacă `b = 0` returnează `a`; altfel `cmmdc(a, b) = cmmdc(b, a % b)`. Definiți o funcție `cmmmc(a, b)` care calculează cel mai mic multiplu comun: `cmmmc(a, b) = a * b / cmmdc(a, b)`.

4 Mediu Definiți următoarele funcții pentru prelucrarea unei liste: (a) `numarare_pozitive(v)` — returnează numărul de elemente pozitive; (b) `suma_patrate(v)` — returnează suma pătratelor elementelor; (c) `elimina_duplicate(v)` — returnează o nouă listă fără duplicate. Citiți o listă și apelați toate cele trei funcții.

Indiciu pentru `elimina_duplicate`: construiți o listă nouă și adăugați elementul curent doar dacă nu se află deja în ea (folosiți operatorul `'in'`).

5 Dificil Definiți o funcție `sortare_bulelor(v)` care sortează crescător o listă folosind algoritmul sortării prin metoda bulelor (Bubble Sort). Algoritmul compară perechi de elemente adiacente și le interschimbă dacă sunt în ordine greșită, repetând procedeul până când lista este sortată.

Indiciu: bucla exterioară `for i in range(n-1)`, bucla interioară `for j in range(n-1-i)`, interschimbare dacă `v[j] > v[j+1]`.

Adăugați un parametru opțional `crescator=True` pentru a sorta și descrescător când `crescator=False`.

- 6** **Dificil** Definiți o funcție `cel_mai_frecvent(v)` care returnează elementul cu cea mai mare frecvență într-o listă. În caz de egalitate, returnați elementul mai mic. Testați funcția pentru lista `[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]`.

Indiciu: pentru fiecare element unic din listă, numărați aparițiile sale cu `v.count(element)`. Rețineți maximum și elementul corespunzător.

Alternativă avansată: folosiți un dicționar pentru frecvențe.

Verifică ce ai învățat

1. Care este diferența dintre o funcție și o procedură? În Python, ce se întâmplă când o funcție nu conține instrucțiunea `return`?
2. Explicați diferența dintre parametri și argumente. Dați un exemplu cu o funcție care are 2 parametri.
3. Ce este o variabilă locală? De ce o variabilă locală nu poate fi accesată în afara funcției în care a fost definită?
4. Când este necesară declarația `global x` în interiorul unei funcții? De ce este considerată o practică ce trebuie evitată?
5. Definiți o funcție `medie(lista)` care primește o listă de numere și returnează media aritmetică. Apelați-o și afișați rezultatul.
6. Ce este o funcție recursivă? Explicați prin exemplul factorialului de ce este obligatoriu să existe un caz de bază (condiție de oprire).